Marzo 2026

Métodos Numéricos

Problemario T3-E2

Ana Isis Garcia de Lucio

Introducción

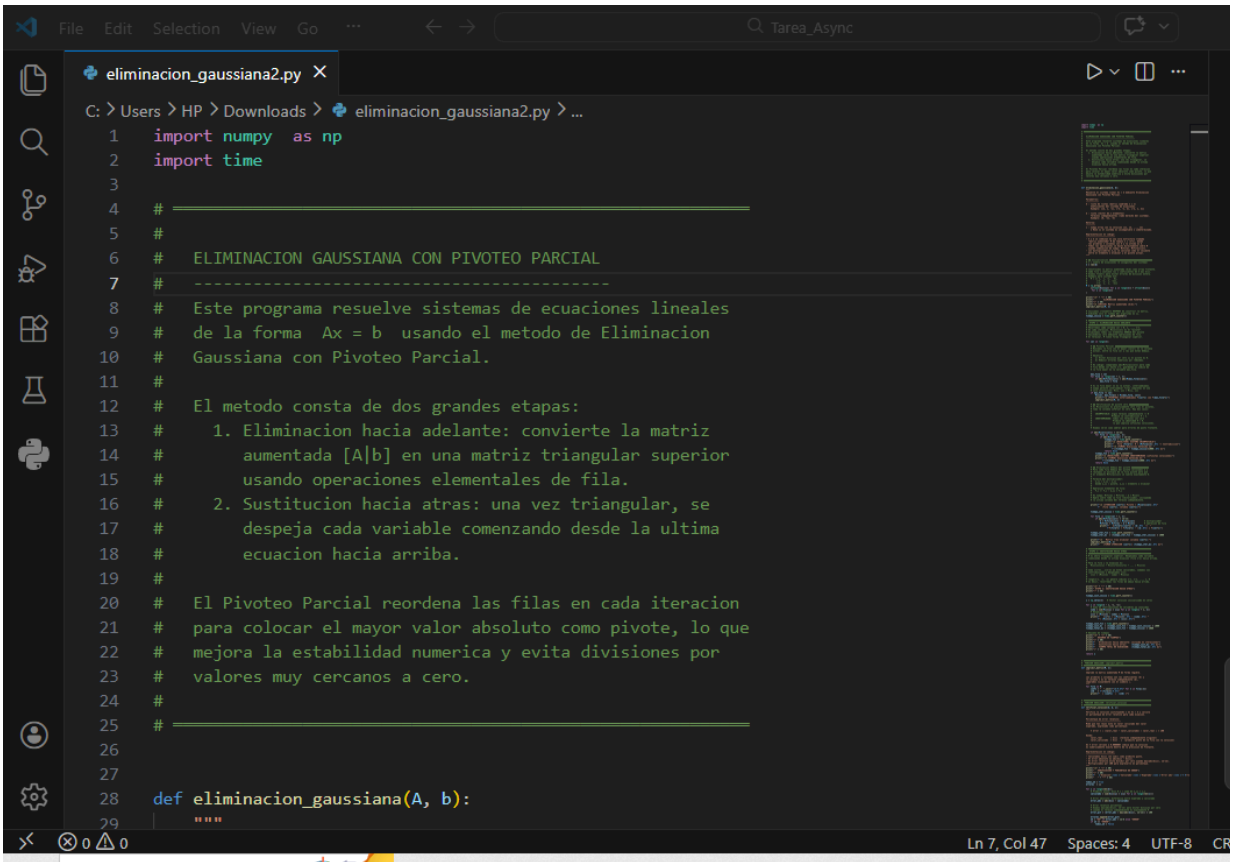
En este trabajo se abordan cuatro métodos importantes para la solución de sistemas de ecuaciones lineales: la Eliminación Gaussiana, el método de Gauss-Jordan, el método de Gauss-Seidel y el método de Jacobi. Los dos primeros son métodos directos que permiten obtener la solución exacta mediante transformaciones de la matriz, mientras que los dos últimos son métodos iterativos que aproximan la solución a través de repeticiones sucesivas.

Eliminación Gaussiana

Algoritmo:

- Escribir el sistema de ecuaciones en forma de matriz aumentada.
- Aplicar operaciones elementales por filas para convertir la matriz en una matriz triangular superior:
- Seleccionar un pivote (elemento principal de la fila).
- Hacer ceros debajo del pivote mediante restas de filas.
- Repetir el proceso para cada columna hasta obtener una matriz triangular.
- Resolver el sistema por sustitución hacia atrás:
- Despejar la última variable.
- Sustituir en las ecuaciones anteriores hasta encontrar todas las incógnitas.

Código



File Edit Selection View Go ··· Tarea_Async

eliminacion_gaussiana2.py

C: > Users > HP > Downloads > eliminacion_gaussiana2.py > ...

```python

def eliminacion_gaussiana(A, b):
    """
    Resuelve el sistema lineal Ax = b mediante Eliminacion
    Gaussiana con Pivoteo Parcial.

    Parametros:
    -----------
    A : lista de listas (matriz cuadrada n x n)
        Coeficientes del sistema de ecuaciones.
        Ejemplo: [[2, 1, -1], [-3, -1, 2], [-2, 1, 2]]

    b : lista (vector de n elementos)
        Terminos independientes (lado derecho del sistema).
        Ejemplo: [8, -11, -3]

    Retorna:
    --------
    x : numpy array con la solucion [x1, x2, ..., xn]
        o None si el sistema es incompatible o indeterminado.

    Representacion en codigo:
    -------------------------
    - A y b se combinan en una sola estructura llamada
      "matriz aumentada" M de tamaño n x (n+1), donde
      las primeras n columnas son A y la ultima es b.
    - Todas las operaciones se hacen directamente sobre M
      usando indexacion de numpy: M[fila], M[fila][col].
    - Los multiplicadores m_ij se calculan como el cociente
```

File Edit Selection View Go ··· Tarea_Async

eliminacion_gaussiana2.py

C: > Users > HP > Downloads > eliminacion_gaussiana2.py > ...

```python
def eliminacion_gaussiana(A, b):


    # ── Inicializacion ──────────────────────────────────────
    # n = numero de ecuaciones (e incognitas del sistema)
    n = len(b)

    # Construimos la matriz aumentada [A|b] como array flotante.
    # Cada fila i contiene los n coeficientes de A mas b[i].
    # Usamos float() para evitar errores de division entera.
    # Ejemplo para sistema 3x3:
    #   M = [[ 2,  1, -1,   8],
    #        [-3, -1,  2, -11],
    #        [-2,  1,  2,  -3]]
    M = np.array(
        [[float(A[i][j]) for j in range(n)] + [float(b[i])]
         for i in range(n)]
    )

    print("\n" + "=" * 60)
    print("        ELIMINACION GAUSSIANA CON PIVOTEO PARCIAL")
    print("=" * 60)
    print("\n [INICIO] Matriz aumentada [A|b]:")
    imprimir_matriz(M, n)

    # Iniciamos cronometro DESPUES de construir la matriz,
    # midiendo solo el tiempo del algoritmo en si.
    tiempo_inicio = time.perf_counter()

```

⊗ 0 ⚠ 0 Ln 7, Col 47 Spaces: 4 UTF-8 CR

File Edit Selection View Go ··· Tarea_Async

eliminacion_gaussiana2.py

C: > Users > HP > Downloads > eliminacion_gaussiana2.py > ...

```python
def eliminacion_gaussiana(A, b):

    # ═══════════════════════════════════════════════════
    #  ETAPA 1: ELIMINACION HACIA ADELANTE
    # ═══════════════════════════════════════════════════
    # Recorremos cada columna col = 0, 1, ..., n-1.
    # En cada iteracion, M[col][col] es el "pivote".
    # Eliminamos todos los elementos DEBAJO del pivote
    # haciendolos cero mediante operaciones de fila.
    # Al terminar, M tiene forma triangular superior.

    for col in range(n):

        # ── Pivoteo Parcial ──────────────────────────
        # Buscamos la fila con el mayor |valor| en la columna
        # actual, entre la fila col y las que estan debajo.
        #
        # Objetivo:
        #   a) Evitar division por cero si el pivote es 0.
        #   b) Reducir errores numericos por redondeo.
        #
        # En codigo: comparamos abs(M[fila][col]) para cada
        # fila desde col hasta n-1, guardando el indice de
        # la fila mayor en la variable max_fila.

        max_fila = col
        for fila in range(col + 1, n):
            if abs(M[fila][col]) > abs(M[max_fila][col]):
                max_fila = fila
```

⊗ 0 ⚠ 0 Ln 7, Col 47 Spaces: 4 UTF-8

File Edit Selection View Go ··· Tarea_Async

eliminacion_gaussiana2.py

C: > Users > HP > Downloads > eliminacion_gaussiana2.py > ...

```python
def menu_principal():
        elif opcion == "3":
            A = [[1, 1], [2, 2]]
            b = [4, 8]
            sol = eliminacion_gaussiana(A, b)
            if sol is None:
                print("\n   El sistema tiene infinitas soluciones.")

        elif opcion == "4":
            A, b, variables = ingresar_sistema()
            sol = eliminacion_gaussiana(A, b)
            if sol is not None:
                print("\n [SOLUCION]")
                for var, val in zip(variables, sol):
                    print(f"   {var} = {val:.6f}")
                verificar_solucion(A, b, sol)

        elif opcion == "0":
            print("\n Hasta luego!\n")
            break
        else:
            print("   Opcion no valida, intenta de nuevo.")

        input("\n  Presiona Enter para continuar...")


if __name__ == "__main__":
    menu_principal()
```

⊗ 0 ⚠ 0 Ln 7, Col 47 Spaces: 4 UTF-8

Eliminación Gaussiana

Ejercicio 1

```
==========================================================
        ELIMINACION GAUSSIANA CON PIVOTEO PARCIAL
==========================================================

 [INICIO] Matriz aumentada [A|b]:
  [    2.0000     1.0000    -1.0000  |     8.0000 ]
  [   -3.0000    -1.0000     2.0000  |   -11.0000 ]
  [   -2.0000     1.0000     2.0000  |    -3.0000 ]

 [PIVOTEO] Intercambiamos F1 con F2
  [   -3.0000    -1.0000     2.0000  |   -11.0000 ]
  [    2.0000     1.0000    -1.0000  |     8.0000 ]
  [   -2.0000     1.0000     2.0000  |    -3.0000 ]

 [ITERACION 1] Pivote = -3.000000  (fila 1, columna 1)
   m21 = -0.666667  ->  F2 = F2 - (-0.666667) x F1
   m31 = 0.666667  ->  F3 = F3 - (0.666667) x F1

   Matriz tras eliminar columna 1:
  [   -3.0000    -1.0000     2.0000  |   -11.0000 ]
  [    0.0000     0.3333     0.3333  |     0.6667 ]
  [    0.0000     1.6667     0.6667  |     4.3333 ]
   [TIMER ITERACION 1] 0.383300 ms

 [PIVOTEO] Intercambiamos F2 con F3
  [   -3.0000    -1.0000     2.0000  |   -11.0000 ]
  [    0.0000     1.6667     0.6667  |     4.3333 ]
  [    0.0000     0.3333     0.3333  |     0.6667 ]

 [ITERACION 2] Pivote = 1.666667  (fila 2, columna 2)
   m32 = 0.200000  ->  F3 = F3 - (0.200000) x F2

   Matriz tras eliminar columna 2:
```

```
   Matriz tras eliminar columna 2:
  [   -3.0000    -1.0000     2.0000  |   -11.0000 ]
  [    0.0000     1.6667     0.6667  |     4.3333 ]
  [    0.0000     0.0000     0.2000  |    -0.2000 ]
   [TIMER ITERACION 2] 0.119700 ms

 [ITERACION 3] Pivote = 0.200000  (fila 3, columna 3)

   Matriz tras eliminar columna 3:
  [   -3.0000    -1.0000     2.0000  |   -11.0000 ]
  [    0.0000     1.6667     0.6667  |     4.3333 ]
  [    0.0000     0.0000     0.2000  |    -0.2000 ]
   [TIMER ITERACION 3] 0.006400 ms

----------------------------------------------------------
 ETAPA 2: SUSTITUCION HACIA ATRAS
----------------------------------------------------------
   x3 = (-0.200000 - 0.000000) / 0.200000 = -1.000000
   x2 = (4.333333 - -0.666667) / 1.666667 = 3.000000
   x1 = (-11.000000 - -5.000000) / -3.000000 = 2.000000

==========================================================
   RESUMEN DE TIEMPOS
==========================================================
   Eliminacion hacia adelante: incluida en iteraciones
   Sustitucion hacia atras:    0.756900 ms
   TIEMPO TOTAL DE EJECUCION:  6.467100 ms
==========================================================

 [SOLUCION]
```

File Edit Selection View Go ···

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
   x2 = (4.333333 - -0.666667) / 1.666667 = 3.000000
   x1 = (-11.000000 - -5.000000) / -3.000000 = 2.000000

============================================================
   RESUMEN DE TIEMPOS
============================================================
   Eliminacion hacia adelante: incluida en iteraciones
   Sustitucion hacia atras:    0.756900 ms
   TIEMPO TOTAL DE EJECUCION:  6.467100 ms
============================================================

 [SOLUCION]
   x = 2.000000
   y = 3.000000
   z = -1.000000

============================================================
   VERIFICACION Y PORCENTAJE DE ERROR
============================================================
   Ecuacion        Calculado     Esperado    Error abs      % Error  Estado
   ------------------------------------------------------------------
   Ec. 1            8.000000     8.000000     0.00e+00    0.000000%  [OK]
   Ec. 2          -11.000000   -11.000000     0.00e+00    0.000000%  [OK]
   Ec. 3           -3.000000    -3.000000     8.88e-16    0.000000%  [OK]
   ------------------------------------------------------------------
   Error maximo:   2.960595e-14%
   Error promedio: 9.868649e-15%

   Solucion verificada correctamente.
   El % de error cercano a 0 confirma precision numerica.
============================================================

  Presiona Enter para continuar...
```

Ejercicio 2

```
==========================================================
=
        ELIMINACION GAUSSIANA CON PIVOTEO PARCIAL
==========================================================
=

 [INICIO] Matriz aumentada [A|b]:
  [    1.0000     1.0000  |     2.0000 ]
  [    2.0000     2.0000  |     5.0000 ]

 [PIVOTEO] Intercambiamos F1 con F2
  [    2.0000     2.0000  |     5.0000 ]
  [    1.0000     1.0000  |     2.0000 ]

 [ITERACION 1] Pivote = 2.000000  (fila 1, columna 1)
   m21 = 0.500000  ->  F2 = F2 - (0.500000) x F1

   Matriz tras eliminar columna 1:
  [    2.0000     2.0000  |     5.0000 ]
  [    0.0000     0.0000  |    -0.5000 ]
   [TIMER ITERACION 1] 0.155600 ms

 [RESULTADO] SISTEMA INCOMPATIBLE
   Fila 2: 0 = -0.500000 -> Contradiccion

 [TIMER] Ejecucion detenida en: 1.152800 ms

   El sistema no tiene solucion.

  Presiona Enter para continuar...
```

Ejercicio 3

```
 Opcion: 3

==============================================================
=
        ELIMINACION GAUSSIANA CON PIVOTEO PARCIAL
==============================================================
=

 [INICIO] Matriz aumentada [A|b]:
  [    1.0000     1.0000  |     4.0000 ]
  [    2.0000     2.0000  |     8.0000 ]

 [PIVOTEO] Intercambiamos F1 con F2
  [    2.0000     2.0000  |     8.0000 ]
  [    1.0000     1.0000  |     4.0000 ]

 [ITERACION 1] Pivote = 2.000000  (fila 1, columna 1)
   m21 = 0.500000  ->  F2 = F2 - (0.500000) x F1

   Matriz tras eliminar columna 1:
  [    2.0000     2.0000  |     8.0000 ]
  [    0.0000     0.0000  |     0.0000 ]
   [TIMER ITERACION 1] 0.429800 ms

 [RESULTADO] SISTEMA INDETERMINADO (infinitas soluciones)

 [TIMER] Ejecucion detenida en: 3.327700 ms

   El sistema tiene infinitas soluciones.

  Presiona Enter para continuar...
```

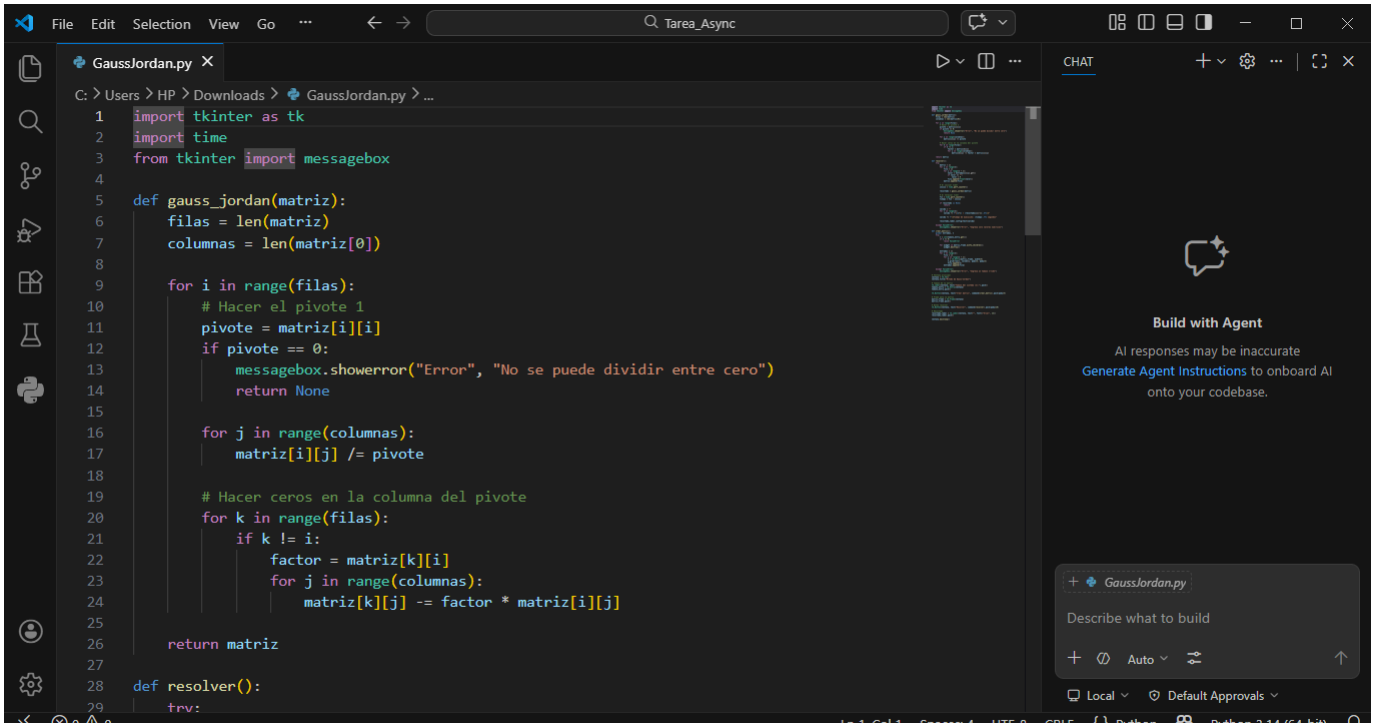
Ln 7, Col 47

Método de Gauss-Jordan

Algoritmo:

1. Escribir el sistema como matriz aumentada.
2. Aplicar operaciones elementales por filas para convertir la matriz en una matriz identidad:
   - Hacer que el pivote sea igual a 1.
   - Hacer ceros arriba y abajo del pivote.
3. Repetir el proceso en cada columna hasta que la matriz del sistema sea la identidad.
4. La solución se obtiene directamente de la última columna de la matriz.

Código



File Edit Selection View Go

Tarea_Async

GaussJordan.py

C: > Users > HP > Downloads > GaussJordan.py > ...

```python

def resolver():
    try:
        matriz = []
        for i in range(n):
            fila = []
            for j in range(n + 1):
                valor = entradas[i][j].get()
                if valor == "":
                    valor = 0
                fila.append(float(valor))
            matriz.append(fila)

        # ⏱ iniciar timer
        inicio = time.perf_counter()

        resultado = gauss_jordan(matriz)

        # ⏱ terminar timer
        fin = time.perf_counter()
        tiempo = fin - inicio

        if resultado is None:
            return

        salida = ""
        for i in range(n):
            salida += f"x{i+1} = {resultado[i][-1]:.3f}\n"

```

CHAT

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

GaussJordan.py

Describe what to build

Auto

Local

Default Approvals

File Edit Selection View Go

Tarea_Async

GaussJordan.py

C: > Users > HP > Downloads > GaussJordan.py > ...

```python
def resolver():
        salida += f"\nTiempo de ejecución: {tiempo:.7f} segundos"

        resultado_label.config(text=salida)

    except ValueError:
        messagebox.showerror("Error", "Ingresa solo valores numéricos")

def crear_matriz():
    global entradas, n
    try:
        n = int(tamano_entry.get())
        if n <= 0:
            raise ValueError

        for widget in matriz_frame.winfo_children():
            widget.destroy()

        entradas = []
        for i in range(n):
            fila = []
            for j in range(n + 1):
                e = tk.Entry(matriz_frame, width=5)
                e.grid(row=i, column=j, padx=3, pady=3)
                fila.append(e)
            entradas.append(fila)

    except ValueError:
```

CHAT

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

GaussJordan.py

Describe what to build

Auto

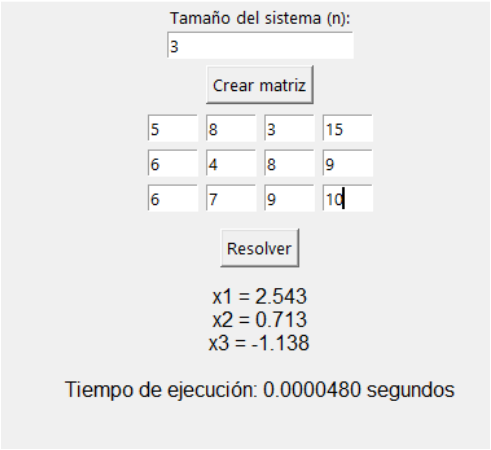
Local

Default Approvals

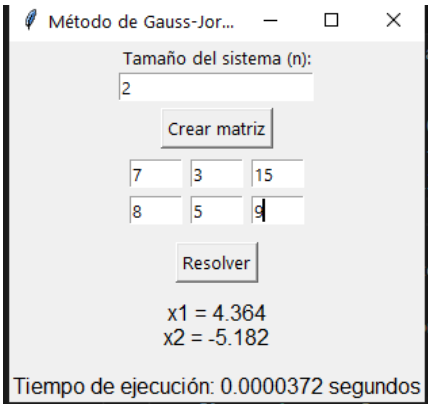
0 0 Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit)

```python
            entradas.append(fila)

        except ValueError:
            messagebox.showerror("Error", "Ingresa un tamaño válido")

# Ventana principal
ventana = tk.Tk()
ventana.title("Método de Gauss-Jordan")

# Tamaño de la matriz
tk.Label(ventana, text="Tamaño del sistema (n):").pack()
tamano_entry = tk.Entry(ventana)
tamano_entry.pack()

tk.Button(ventana, text="Crear matriz", command=crear_matriz).pack(pady=5)

# Frame para la matriz
matriz_frame = tk.Frame(ventana)
matriz_frame.pack()

# Botón resolver
tk.Button(ventana, text="Resolver", command=resolver).pack(pady=10)

# Resultado
resultado_label = tk.Label(ventana, text="", font=("Arial", 12))
resultado_label.pack()

ventana.mainloop()
```

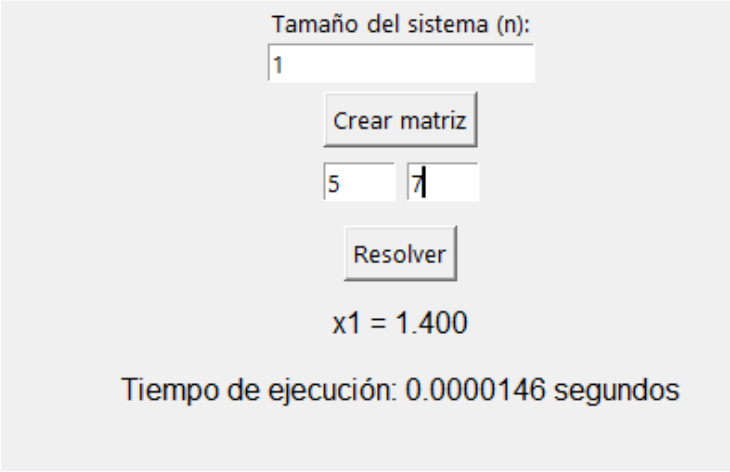
Ejercicio 1



Ejercicio 2



Ejercicio 2



Método de Gauss-Seidel

Algoritmo:

- Despejar cada variable en función de las demás ecuaciones.
- Elegir valores iniciales para las incógnitas (por ejemplo, o).
- Sustituir los valores en las ecuaciones despejadas para obtener nuevas aproximaciones:
- Usar inmediatamente los valores recién calculados en la misma iteración.
- Repetir el proceso iterativo hasta que el error entre dos iteraciones consecutivas sea menor a una tolerancia establecida.

Código

```python
import numpy as np

def gauss_seidel(A, b, tolerance=1e-5, max_iterations=100):
    """
    Resuelve el sistema Ax = b usando el método de Gauss-Seidel.
    """
    n = len(b)
    # Vector inicial de ceros
    x = np.zeros_like(b, dtype=np.double)

    for k in range(max_iterations):
        x_old = x.copy()

        for i in range(n):
            # Suma de la parte ya actualizada (j < i)
            sum1 = sum(A[i][j] * x[j] for j in range(i))
            # Suma de la parte de la iteración anterior (j > i)
            sum2 = sum(A[i][j] * x_old[j] for j in range(i + 1, n))

            # Cálculo del nuevo valor para x_i
            x[i] = (b[i] - sum1 - sum2) / A[i][i]

        # Calcular el error (norma euclidiana)
        error = np.linalg.norm(x - x_old, ord=np.inf)

        # Verificar convergencia
        if error < tolerance:
            print(f"Convergió en {k+1} iteraciones.")
            return x

    print("El método no convergió en el número máximo de iteraciones.")
    return x

# --- Ejemplo de uso ---
A = np.array([[4, 1, 2],
              [3, 5, 1],
              [1, 1, 3]], dtype=float)
b = np.array([4, 7, 3], dtype=float)

solucion = gauss_seidel(A, b)
print("Solución:", solucion)
```

Validación

```
Convergió en 9 iteraciones.
Solución: [0.50000128 0.99999949 0.49999974]
```

Método de Jacobi

Algoritmo:

- Despejar cada variable en función de las demás.
- Asignar valores iniciales a todas las incógnitas.
- Calcular nuevos valores sustituyendo los valores anteriores:
- No usar valores actualizados en la misma iteración (todos se actualizan al final).
- Repetir el proceso hasta que el error sea suficientemente pequeño o se alcance un número máximo de iteraciones.

Código

Tarea_Async

Sin título1.py | Jacobi.py

C: > Users > HP > Downloads > Jacobi.py > ...

```python
# Importa la librería tkinter para crear interfaces gráfic
import tkinter as tk

# Importa las ventanas de mensajes (errores, alertas, etc.)
from tkinter import messagebox


# ---------------- MÉTODO JACOBI ----------------

# Función que implementa el método iterativo de Jacobi
def jacobi(A, b, x0, tol, max_iter):

    # Obtiene el número de ecuaciones (tamaño del vector b)
    n = len(b)

    # Copia los valores iniciales x0 al vector x
    x = x0[:]

    # Limpia el cuadro de resultados antes de comenzar
    resultado.delete("1.0", tk.END)

    # Ciclo de iteraciones hasta el máximo permitido
    for k in range(max_iter):

        # Crea un nuevo vector para guardar los nuevos val
        x_new = [0]*n

        # Recorre cada ecuación
        for i in range(n):
```

File Edit Selection View Go ... Tarea_Async

EXPLORER

TAREA_ASYNC
- hi.py
- hola.py
- python_asincrono.py
- python_sincrono.py
- um.py

OUTLINE
TIMELINE

Sin título1.py | Jacobi.py

C: > Users > HP > Downloads > Jacobi.py > ...

```python
# Entrada para el vector b
entry_b = tk.Entry(ventana, width=40, justify="center")
entry_b.pack(pady=5)


# Texto para ingresar los valores iniciales
tk.Label(ventana, text="Valores iniciales x0:", bg="#1e

# Entrada para x0
entry_x0 = tk.Entry(ventana, width=40, justify="center")
entry_x0.pack(pady=5)


# Texto para la tolerancia
tk.Label(ventana, text="Tolerancia:", bg="#1e1e2f", fg=

# Entrada para tolerancia
entry_tol = tk.Entry(ventana, justify="center")
entry_tol.pack(pady=5)


# Texto para iteraciones máximas
tk.Label(ventana, text="Máximo de iteraciones:", bg="#1

# Entrada para iteraciones
entry_iter = tk.Entry(ventana, justify="center")
entry_iter.pack(pady=5)
```

CHAT

Build with Agent

AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.

Jacobi.py

Describe what to build

Auto

Local | Default Approvals

Tarea_Async

Sin título1.py | Jacobi.py

C: > Users > HP > Downloads > Jacobi.py > ...

```python

# Botón que ejecuta el método de Jacobi
tk.Button(
    ventana,
    text="Ejecutar Jacobi",
    command=ejecutar,
    bg="#00ff88",
    fg="black",
    font=("Arial", 12, "bold")
).pack(pady=10)


# Caja de texto donde se mostrarán los resultados
resultado = tk.Text(
    ventana,
    height=12,
    width=90,
    bg="#0f0f1a",
    fg="#00ff88"
)

# Coloca la caja de resultados en la ventana
resultado.pack(pady=10)


# Inicia el ciclo principal de la interfaz gráfica
ventana.mainloop()
```

Ejercicio 1

Ejercicio 2

Ejercicio 3

Método de Jacobi - Profesional
Método de Jacobi
Número de ecuaciones:
2
Crear Matriz
Fila 1
2
5
Fila 2
-3
7
Vector b (separado por espacios):
5 36
Valores iniciales x0:
0 0
Tolerancia:
0.0001
Máximo de iteraciones:
50
Ejecutar Jacobi
Iteración 44: [17.811871611066007, -10.687122966639606] Error: 54.748491866558425
Iteración 45: [29.217807416599015, 12.776516404742575] Error: 23.46363937138218
Iteración 46: [-29.441291011856435, 17.664774607113863] Error: 58.65909842845545
Iteración 47: [-41.66193651778465, -7.474839005081329] Error: 25.13961361219519
Iteración 48: [21.187097512703325, -12.712258507621994] Error: 62.84903403048798
Iteración 49: [34.280646269054984, 14.223041791158566] Error: 26.93530029878056
Iteración 50: [-33.05760447789642, 19.83456268673785] Error: 67.3382507469514
No convergió
Solución aproximada: [-33.05760447789642, 19.83456268673785]

Ana Isis Garcia de Lucio